Birla Institute of Technology and Science, Pilani

Work Integrated Learning Programme (WILP)

Machine Learning - Assignment 2 (AIML)

Mobile Price Classification - Multi-class Models - Streamlit Deployment

Student Name: Vedant Chaudhari
BITS Pilani Student ID: 2026aj05282

Table of Contents

Sec.	Topic	Pages
1-2	GitHub Repository and Live Streamlit Application	2
3	Evidence Screenshots (Virtual Lab and Application)	3-6
4	README Documentation (a) through (d)	7+

1. GitHub Repository

GitHub Repository
https://github.com/vedant237/AIML-ML-Assignment-2

2. Live Streamlit Application

Hosted Streamlit Application
https://aiml-ml-assignment-2-7zp9qjtttffmzswfxtrdzb.streamlit.app/

3. Evidence Screenshots (Virtual Lab and Application)

Figures 1-7 show the Virtual Lab workflow, five single-model Streamlit evaluations on test_data.csv (500 records), and the Streamlit Cloud multi-model summary. They document command usage, UI configuration, and comparative results in one submission packet.

Figure 1. Virtual Lab terminal: train_models.py output and streamlit run app.py.

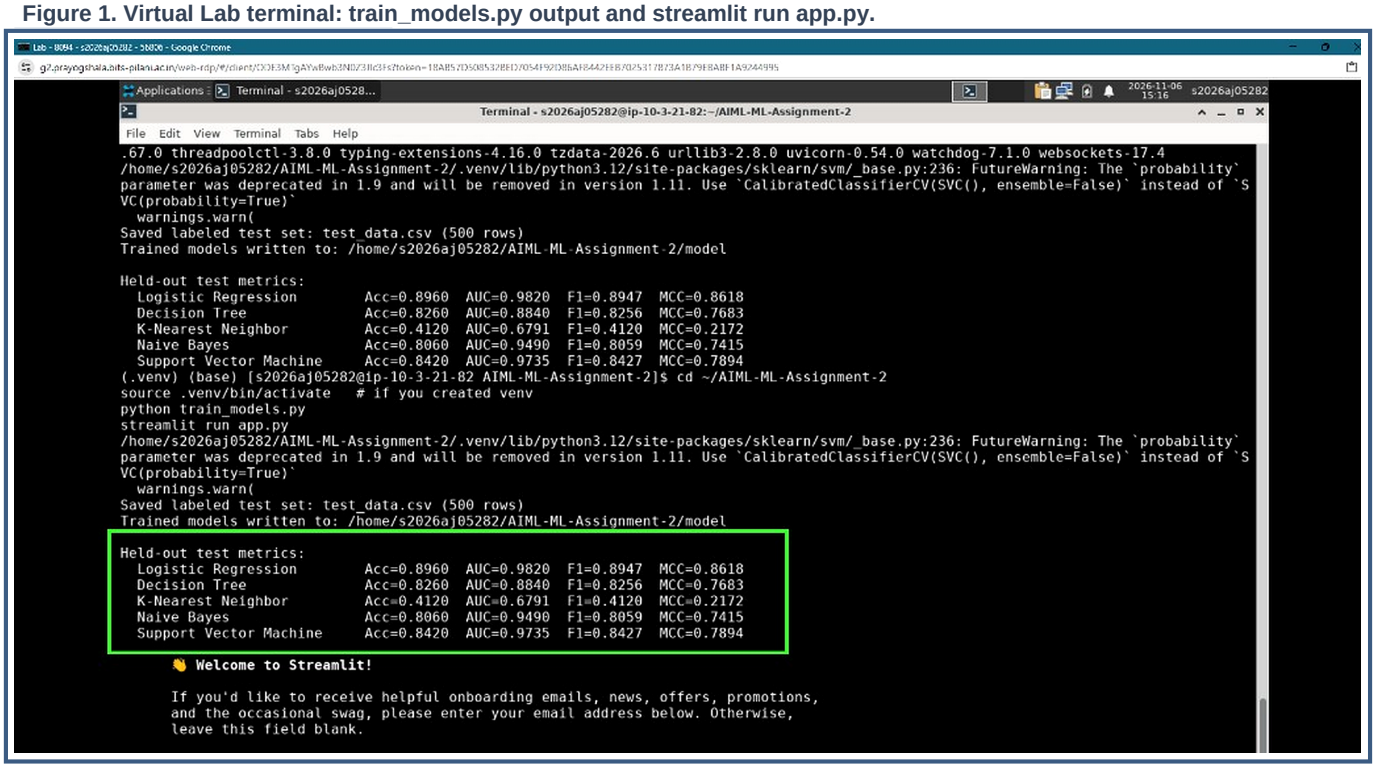


Figure 2. Streamlit (localhost): Logistic Regression; test_data.csv (500 rows); metrics, confusion matrix, classification report.

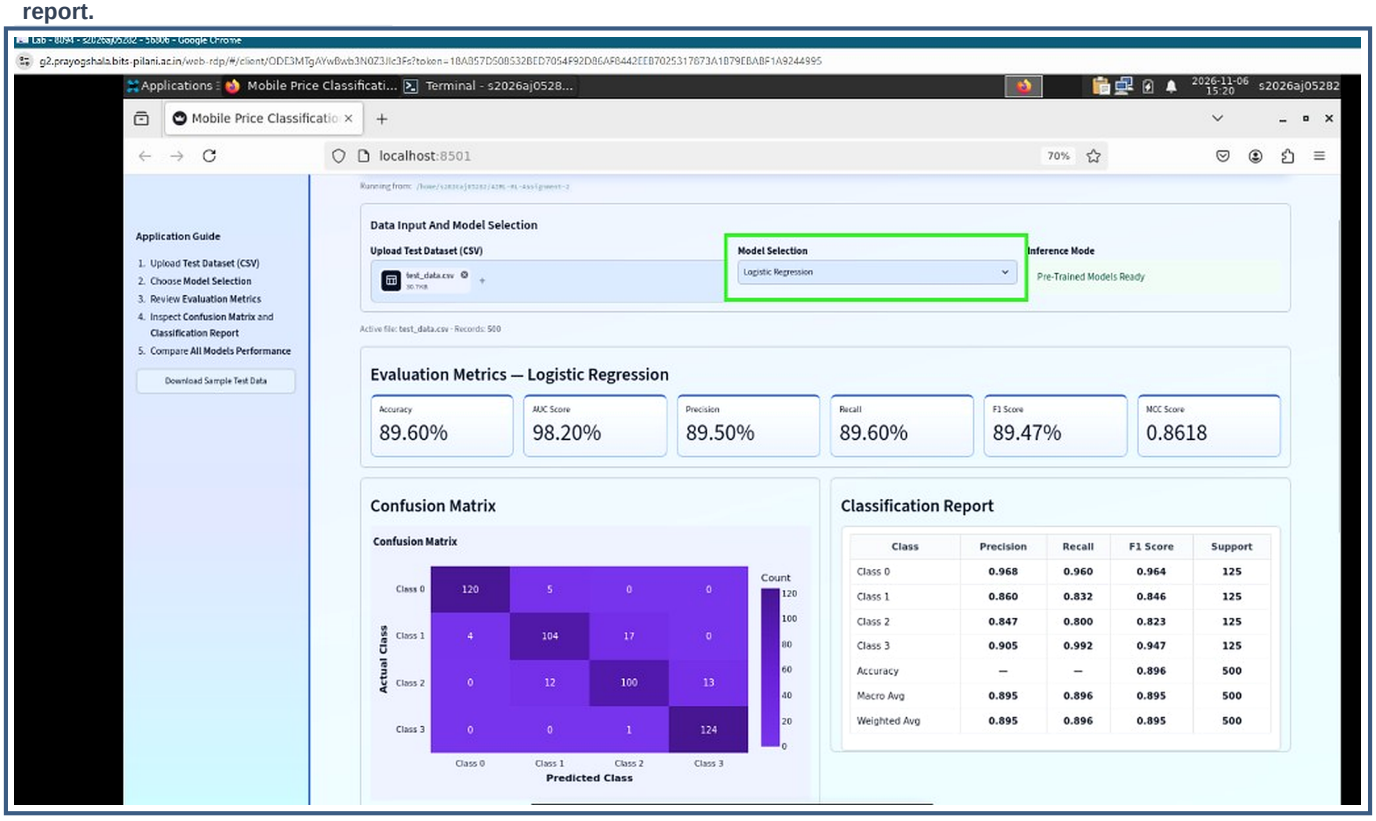


Figure 3. Streamlit (localhost): Decision Tree; same test file and dashboard layout.

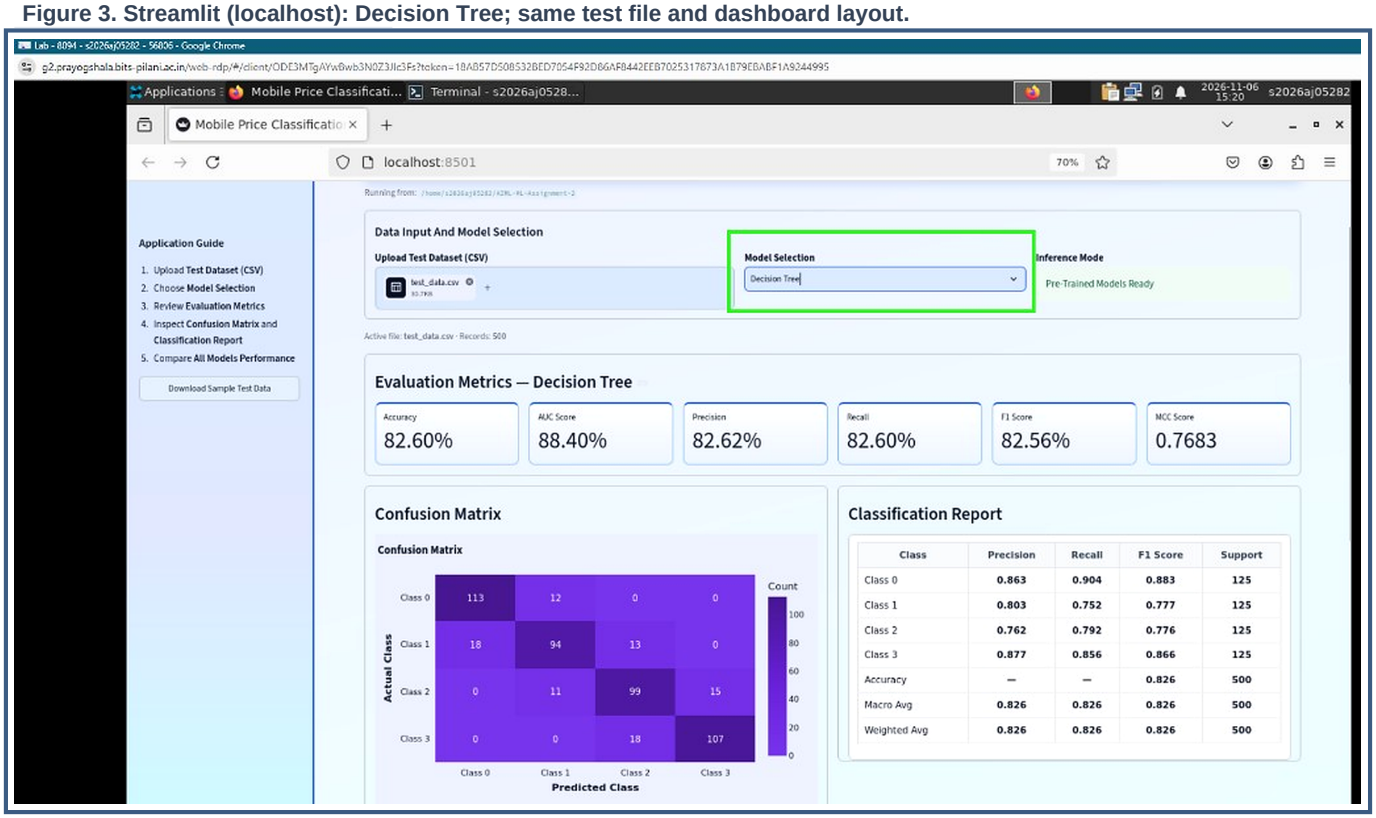


Figure 4. Streamlit (localhost): K-Nearest Neighbor; same test file and dashboard layout.

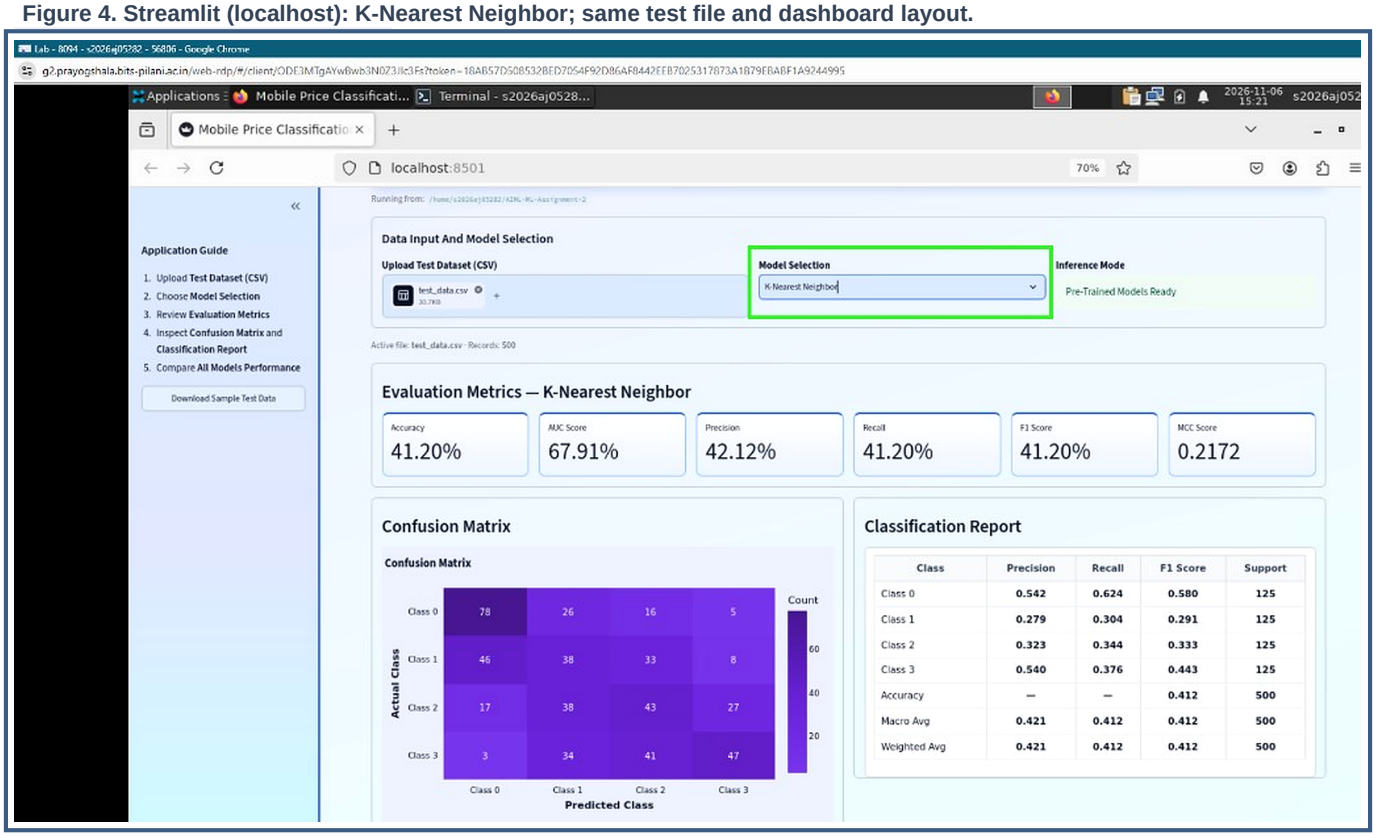


Figure 5. Streamlit (localhost): Naive Bayes (Gaussian); same test file and dashboard layout.

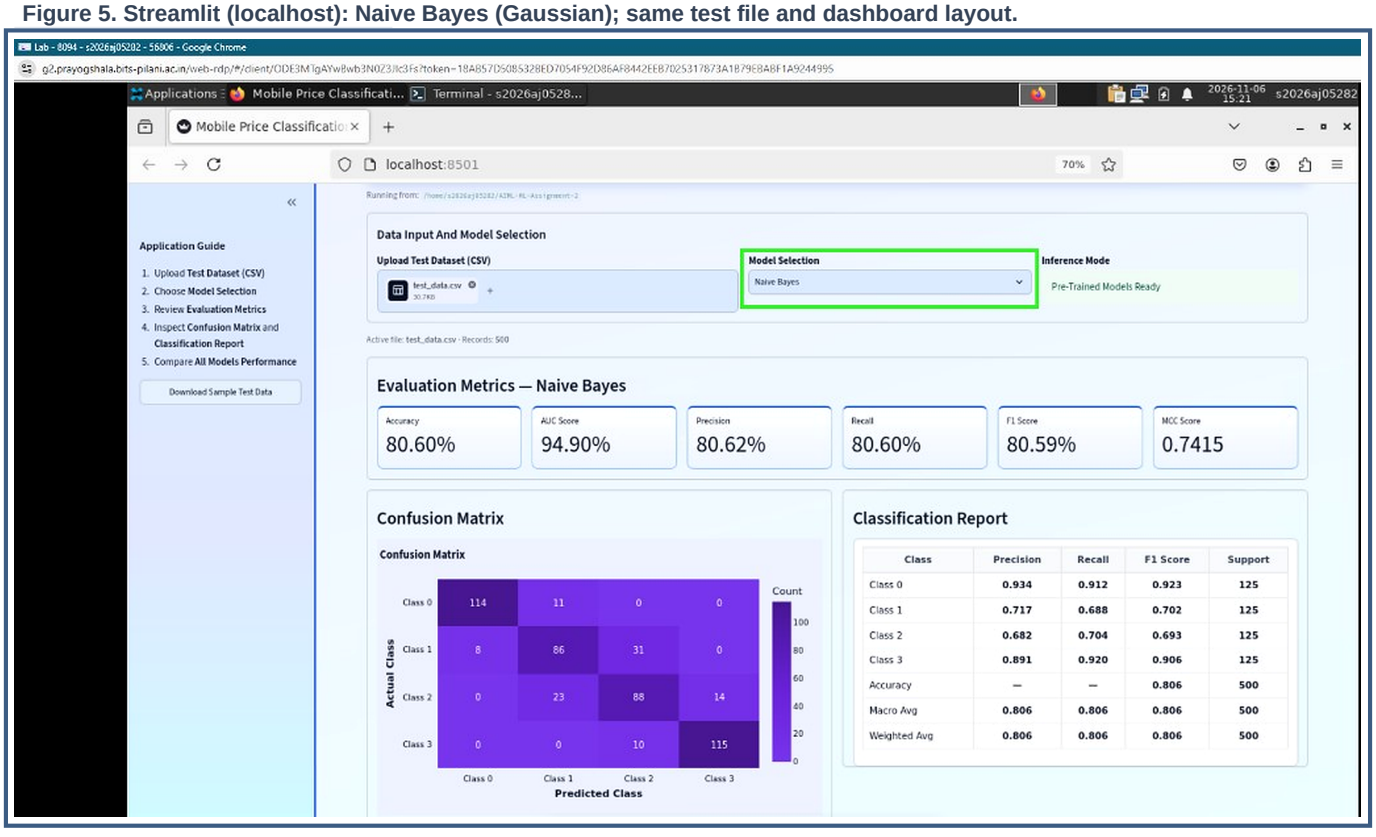


Figure 6. Streamlit (localhost): Support Vector Machine; same test file and dashboard layout.

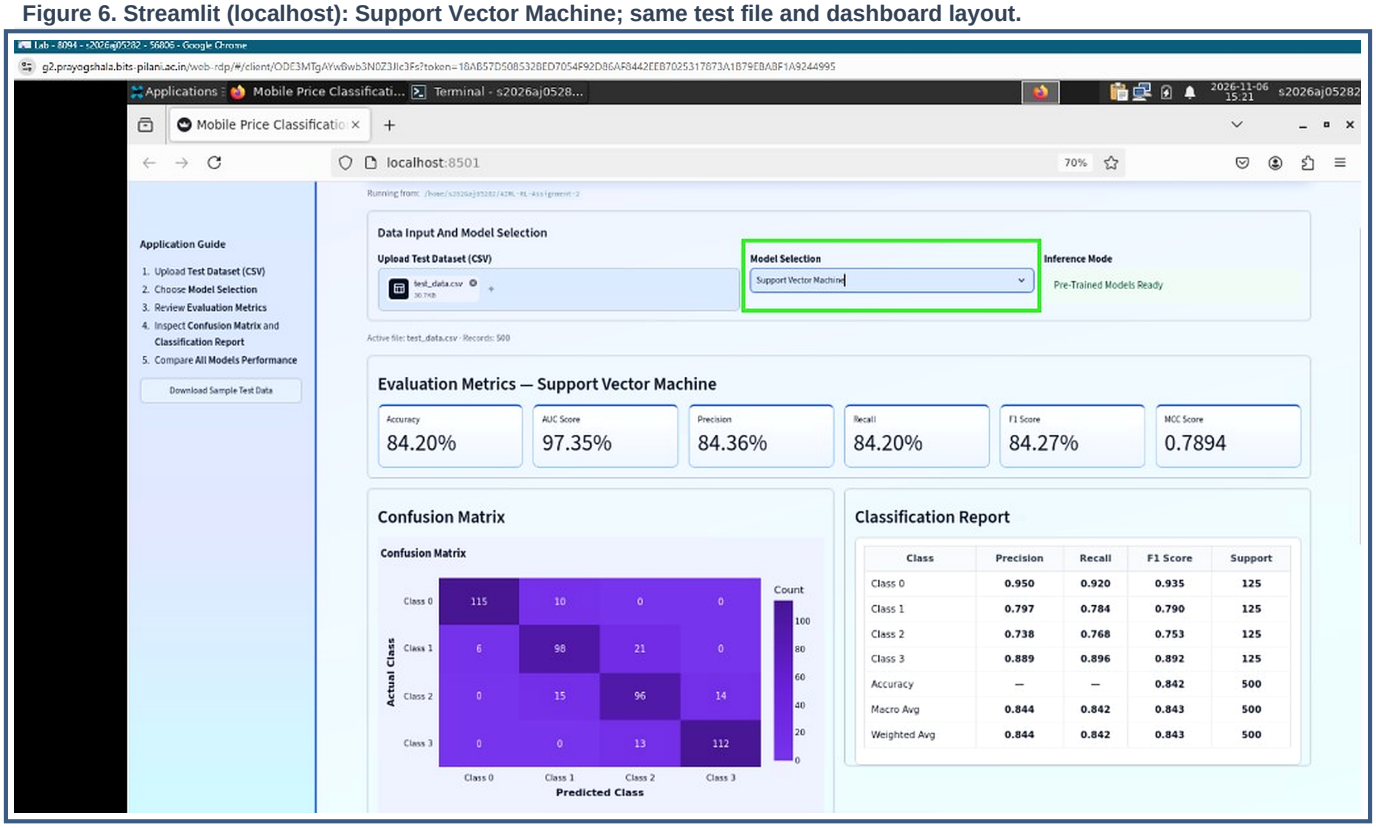
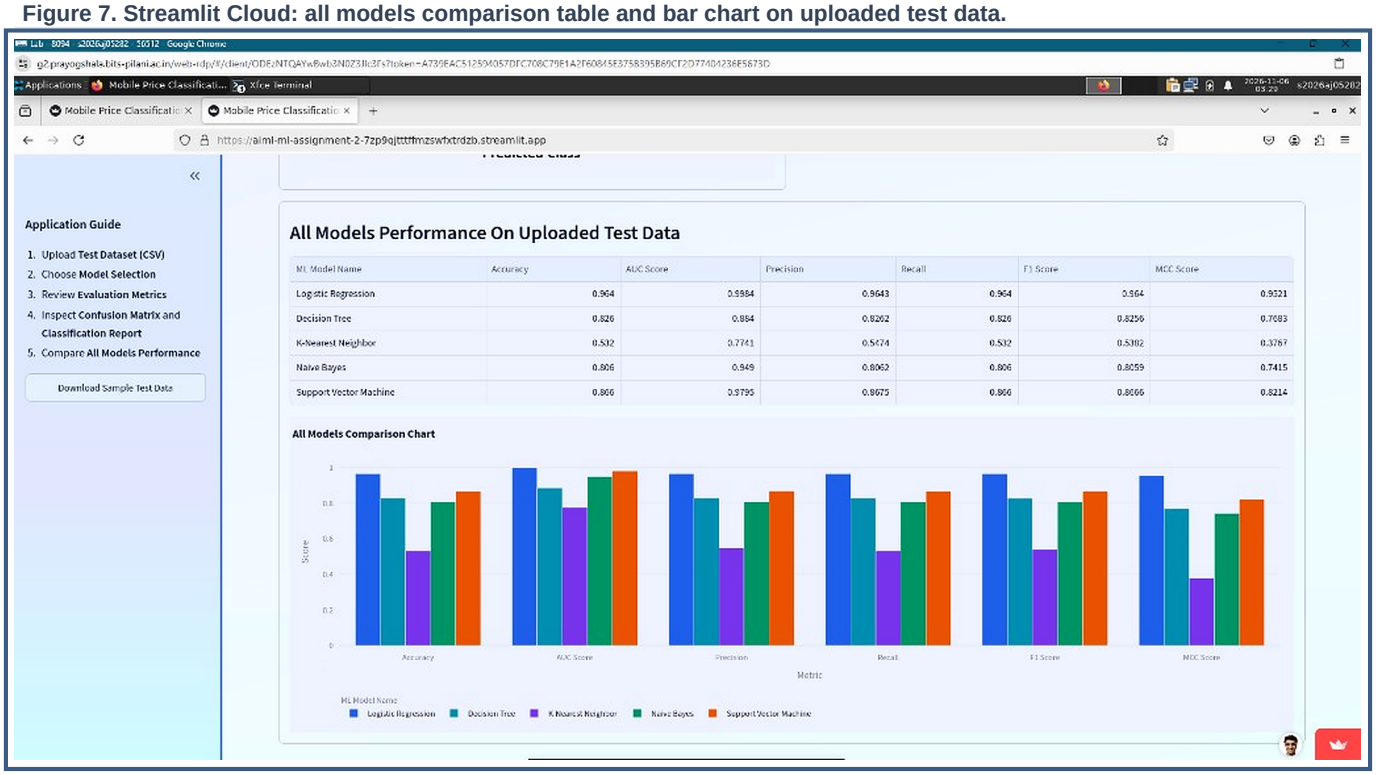


Figure 7. Streamlit Cloud: all models comparison table and bar chart on uploaded test data.



4. README Documentation (Sections a through D)

Sections (a) through (d) below restate the course README committed to the repository: problem statement, dataset specification, repository and application links, and model comparison.

Section (a). Problem Statement

The objective is to predict the price range of a mobile phone using hardware and connectivity attributes. This is a multi-class classification problem with four target classes (0-3).

The project implements five classification algorithms on one dataset, evaluates them with standard metrics, stores trained models in the repository, and exposes an interactive Streamlit application. The deployed app loads saved models and evaluates uploaded labeled test CSV files only; no retraining is performed in the app.

Section (b). Dataset Description

Item	Description
Dataset name	Mobile Price Classification
Repository	Kaggle - Mobile Price Classification (https://www.kaggle.com/datasets/iabhishekofficial/mobile-price-classification)
File in GitHub repo	mobile_price_dataset.csv (training data; renamed from Kaggle train.csv)
Number of instances	2,000
Number of features	20 (numeric): e.g. battery_power, ram, px_height, px_width, mobile_wt
Target variable	price_range - classes 0, 1, 2, 3
Problem type	Multi-class classification
Test file for app	test_data.csv - 500 stratified hold-out rows (25%) with all features and target

Assignment criteria: minimum 12 features and 500 instances - satisfied (20 features, 2,000 instances).

Section (c). Repository and Application Links

GitHub Repository:

https://github.com/vedant237/AIML-ML-Assignment-2

Live Streamlit Application:

https://aiml-ml-assignment-2-7zp9qjtttffmzswfxtrdzb.streamlit.app/

Section (d). Models Used and Evaluation

All models are trained on the same dataset (mobile_price_dataset.csv, 1,500 training rows after split). Evaluation metrics below are computed on the held-out test set (test_data.csv, 500 rows) using train_models.py.

Models implemented:

1. Logistic Regression
2. Decision Tree Classifier
3. K-Nearest Neighbor Classifier
4. Naive Bayes Classifier (Gaussian)
5. Support Vector Machine

Model	Accuracy	AUC	Prec.	Recall	F1	MCC
Logistic Regression	0.9640	0.9984	0.9643	0.9640	0.9640	0.9521
Decision Tree	0.8260	0.8840	0.8262	0.8260	0.8256	0.7683
K-Nearest Neighbor	0.5320	0.7741	0.5474	0.5320	0.5382	0.3767
Naive Bayes	0.8060	0.9490	0.8062	0.8060	0.8059	0.7415
Support Vector Machine	0.8660	0.9795	0.8675	0.8660	0.8666	0.8214

Metrics Note

weighted precision, recall, and F1 where applicable; AUC computed with one-vs-rest (weighted). Source: training_metrics.json.

Observations

Model	Observation
Logistic Regression	**Ranks first on all six metrics on the 500-row hold-out (e.g. accuracy 0.964**, **MCC 0.952**, multiclass **AUC ~0.998**). The **confusion matrix** stays near-diagonal, so mistakes concentrate on adjacent `price_range` bands. With MinMax scaling, linear boundaries track RAM-like and camera-related features well; weighted **precision**, **recall**, and **F1** remain balanced across classes, which supports selecting this pipeline for **deployment** and **Streamlit** scoring.
Decision Tree	**Accuracy** 0.826 **and AUC ~0.88; non-linear splits capture interactions but depth is capped at 12 to limit variance.** Error mass sits between neighbouring classes (notably 1 vs 2), consistent with ordinal confusion between mid-tier phones. The tree is interpretable but trails kernel and linear models on this scaled numeric tabular task.
K-Nearest Neighbor	**Weakest suite (accuracy** 0.532**, MCC ~0.38, AUC ~0.77): kNN with k=5 votes in a 20-feature space where class manifolds overlap, so predictions scatter off the diagonal.** Euclidean distance after scaling still struggles when many attributes correlate; tuning k or distance metrics would be needed to narrow the gap to the top models.
Naive Bayes	**Mid-tier results (accuracy ~0.81, AUC ~0.95) under a Gaussian independence assumption.** Performance holds on low vs high tiers more than on overlapping middle bands, which aligns with correlated specs violating conditional independence. It remains a fast baseline but is outpaced by discriminative models on this dataset.
Support Vector Machine	**Second overall (accuracy** 0.866**, MCC ~0.82, AUC ~0.98) using an RBF kernel and probability estimates for AUC.** Residual errors lean toward middle classes versus **logistic regression**, while ranking quality (**AUC**) stays strong, indicating good separation in score space despite a few more label swaps on the diagonal.
Overall, Winner for your dataset?	**Logistic Regression** delivers the best trade-off of **accuracy**, calibration, and **MCC** on `test_data.csv` under fixed `**random_state=42**`; it is reported as the **recommended model** for this assignment and repository snapshot.